

End-to-End LLM Chatbot with RAG, Monitoring & Deployment

AI & Machine Learning (AIML) – Applied Generative AI / LLMOps Capstone

Project Report

1. Introduction

This project is an End-to-End LLM Chatbot with RAG, Monitoring & Deployment.

The main purpose of this project is to create a chatbot that can answer questions using information from a custom knowledge document.

The chatbot first searches the knowledge document for relevant information and then uses that information to generate the answer. This approach is called Retrieval-Augmented Generation (RAG).

The project also includes monitoring to record chatbot questions, answers, retrieval time, and response latency.

2. Project Objective

The project was developed to demonstrate the complete workflow of a RAG-based chatbot.

- Create a knowledge base using a document.
- Split the document into smaller chunks.
- Convert the chunks into embeddings.
- Store the embeddings in a vector database.
- Retrieve relevant information for a user question.
- Use an LLM to generate an answer.
- Create a simple chatbot interface.
- Monitor chatbot performance.
- Test whether the chatbot stays grounded in the provided documents.
- Deploy and test the chatbot as a web application.

3. Technologies Used

The following technologies were used to build the project:

Python – Main programming language.

Jupyter Notebook – Development and testing.

LangChain – RAG and document processing.

Sentence Transformers – Creating embeddings.

FAISS – Vector database.

Ollama – Running the local LLM.

Llama 3.2 3B – Answer generation.

Streamlit – Chatbot web application.

Pandas – Reading and analyzing logs.

Plotly – Monitoring charts.

GitHub – Storing the project source code.

4. Project Setup

The project was organized into separate folders for documents, vector data, logs, application files, and evaluation work.

Project structure:

LLM_RAG_Chatbot / documents / vector_db / logs / app / evaluation / rag_chatbot.ipynb / requirements.txt.

5. Creating the Knowledge Document

A text knowledge document named `python_notes.txt` was created inside the documents folder.

The document contains basic Python topics such as Python introduction, variables, data types, conditional statements, loops, functions, lists, dictionaries, exception handling, object-oriented programming, and machine learning.

This document became the custom knowledge base for the chatbot.

6. Loading the Document

The `TextLoader` from `LangChain` was used to load `python_notes.txt` into the application.

The document content was then available for further processing.

7. Text Chunking

The document was divided into smaller pieces called chunks using `RecursiveCharacterTextSplitter`.

The chunk size used was 300 characters and the chunk overlap was 50 characters.

Chunking makes it easier to search for only the relevant parts of the document.

8. Creating Embeddings

Each document chunk was converted into a numerical vector called an embedding.

The Sentence Transformer model `sentence-transformers/all-MiniLM-L6-v2` was used for creating the embeddings.

These embeddings allow the system to compare the meaning of a user question with the meaning of document chunks.

9. Creating the FAISS Vector Database

The document chunks and their embeddings were stored in a FAISS vector database.

The FAISS database was saved inside the vector_db folder.

FAISS is used to quickly find document chunks that are semantically similar to a user question.

10. Semantic Search

A semantic search test was performed using questions such as "What are Python data types?"

FAISS searched the stored embeddings and returned the most relevant document chunks.

This retrieved information was then used as the context for the language model.

11. Connecting the LLM

Ollama was used to run the language model locally.

The model used in the project was llama3.2:3b.

The retrieved document information was passed to the model as context so that the model could generate an answer based on the knowledge document.

12. Building the RAG Chatbot

The complete RAG flow is: User Question → Semantic Search → FAISS Vector Database → Relevant Document Chunks → Context → Llama 3.2 3B → Final Answer.

The prompt instructs the model to answer using only the provided context.

If the answer is not available in the document, the chatbot is instructed to respond: "I don't know based on the provided documents."

The chatbot interface was created using Streamlit.

13. Hallucination / Groundedness Test

Two questions were tested.

The question "What are Python data types?" was answered using information from the knowledge document.

The question "Who is the President of India?" was outside the knowledge document. The chatbot returned "I don't know based on the provided documents."

This test demonstrated that the chatbot can avoid answering questions when the required information is not present in the provided documents.

14. Monitoring and Logging

The chatbot records interaction information in logs/chat_logs.csv.

The monitoring information includes timestamp, question, retrieved context, answer, retrieval time, total latency, and feedback.

The chatbot interface also displays retrieval time, generation time, and total latency for a response.

During testing, one response showed retrieval time of 0.19 seconds, generation time of 28.99 seconds, and total latency of 29.18 seconds.

15. Monitoring Dashboard

A monitoring dashboard was created in app/dashboard.py using Streamlit.

The dashboard displays total queries, average latency, token information when available, positive feedback, response performance, user feedback, and interaction logs.

The dashboard was successfully opened and tested.

At the recorded evaluation point, the dashboard showed 1 total query, average latency of 22.19 seconds, 0 total tokens in the available old log data, and 0 positive feedback.

16. Evaluation

Retrieval relevance was tested using multiple questions to check whether relevant document chunks were retrieved.

Groundedness was tested using both an in-document question and an outside-document question.

Latency was measured from the chatbot log.

Feedback values were checked without adding artificial feedback data.

The recorded latency evaluation showed 1 query, average retrieval time of 0.0563 seconds, and average total latency of 22.1874 seconds.

The feedback evaluation showed 0 positive feedback, 0 negative feedback, and 1 feedback value not provided.

17. Deployment

The chatbot was run as a Streamlit web application using the local Ollama model.

The application was tested in a web browser and successfully answered questions from the knowledge base.

The project source code was also uploaded to a GitHub repository.

The implementation demonstrates the chatbot as a working local web application. It was not presented as a cloud-hosted deployment.

18. Conclusion

This project successfully demonstrates an end-to-end LLM chatbot using Retrieval-Augmented Generation.

A custom Python knowledge document was created and loaded into the system. The document was divided into chunks, converted into embeddings, and stored in a FAISS vector database.

When a user asks a question, the system performs semantic search and retrieves relevant document information. The retrieved information is then provided to the Llama 3.2 3B model running through Ollama.

The chatbot was built with Streamlit and tested as a web application. Monitoring was added to record questions, retrieved context, response times, and other available information.

The project was evaluated using retrieval relevance, groundedness, latency, and feedback checks.

Overall, the project provides a practical beginner-level implementation of a RAG-based chatbot, covering the complete flow from document loading and vector search to answer generation, monitoring, evaluation, and web application testing.